PHASE 6 PLANNING DOCUMENT

Enterprise Intelligence and Production Readiness

Six milestones (M29-M34) that transform Probato from a comprehensive testing platform into an intelligent, self-optimizing quality system with production monitoring, enterprise compliance, and extensible plugin architecture.

Probato AI Testing Platform

Milestones 29-34

May 2026

Table of Contents

1. Executive Summary

Phase 6 represents the next evolutionary leap for Probato, transitioning the platform from a comprehensive testing tool into an **intelligent, self-optimizing quality platform** that enterprise teams can rely on for production-grade confidence. After five phases and twenty-eight milestones of foundational construction, Phase 6 introduces the intelligence layer that transforms raw test data into actionable insights, makes tests self-maintaining, extends quality assurance into production monitoring, and delivers the enterprise-grade security and compliance capabilities that large organizations demand.

The six milestones in Phase 6 are organized around three strategic pillars. The first pillar, **AI Intelligence**, encompasses M29 (AI Test Intelligence Engine) and M30 (Self-Healing Tests v2), which together create a feedback loop where the platform learns from every test run, predicts failures before they happen, and automatically repairs broken tests without human intervention. The second pillar, **Production Readiness**, covers M31 (Synthetic Monitoring and Performance Baselines) and M32 (Enterprise SSO, Audit, and Compliance), bridging the gap between pre-deployment testing and post-deployment monitoring while ensuring the platform meets the security and governance requirements of enterprise customers. The third pillar, **Extensibility**, includes M33 (Plugin Architecture and Integrations Marketplace) and M34 (Phase 6 Integration and Polish), which open the platform to community-driven innovation and ensure that all Phase 6 features work seamlessly together.

By the end of Phase 6, Probato will offer a complete quality intelligence loop: discover features, generate tests, execute them intelligently based on risk and impact analysis, self-heal broken selectors, monitor production health, and provide compliance-ready audit trails. This positions Probato as the definitive platform for teams that need to ship faster with higher confidence, from startups to Fortune 500 enterprises.

2. Phase 6 Overview

Phase 6 introduces six milestones (M29 through M34) that collectively transform Probato from a reactive testing platform into a proactive quality intelligence system. The phase builds directly on the multi-device orchestration capabilities established in Phase 5, leveraging the existing data models, API infrastructure, and credit-based billing system to add intelligence, production monitoring, enterprise compliance, and extensibility layers.

Milestone	Name	Category	Est. Credits
M29	AI Test Intelligence Engine	AI Intelligence	20 credits/analysis
M30	Self-Healing Tests v2 and Auto-Maintenance	AI Intelligence	8 credits/repair

M31	Synthetic Monitoring and Performance Baselines	Production Readiness	3 credits/check
M32	Enterprise SSO, Audit, and Compliance	Production Readiness	N/A (plan feature)
M33	Plugin Architecture and Integrations Marketplace	Extensibility	Variable per plugin
M34	Phase 6 Integration and Polish	Cross-cutting	N/A

Table 1: Phase 6 Milestone Overview

3. M29: AI Test Intelligence Engine

The AI Test Intelligence Engine is the brain of Phase 6. It transforms the vast amount of test execution data that Probato already collects into predictive insights and optimization recommendations. Today, teams run tests and review pass/fail results reactively. After M29, Probato will proactively tell teams which tests are most likely to fail, which areas of the application carry the highest risk, and how to optimize test suites for speed and coverage. This milestone introduces three core capabilities: smart test selection, flakiness prediction and classification, and impact-based test prioritization.

3.1 Smart Test Selection

Smart Test Selection analyzes code changes and determines which tests are relevant to run, eliminating the wasteful practice of running the entire test suite on every commit. The system works by building a dependency graph between application code and test cases, mapping which source files, functions, and components each test exercises. When a pull request modifies specific files, the intelligence engine traverses the dependency graph to identify the minimal set of tests that could be affected, reducing test execution time by an estimated 60-80% for most pull requests while maintaining equivalent defect detection rates.

The dependency graph is constructed from multiple signals. First, the LLM code analysis agent already maps features to source code locations during the discovery phase, providing a baseline mapping. Second, test execution traces capture which API endpoints, DOM selectors, and navigation paths each test touches. Third, runtime coverage data from Puppeteer and Playwright executions records the exact source lines exercised by each test. These signals are fused into a weighted bipartite graph stored in the database, where nodes represent tests and code elements, and edge weights represent the strength of the relationship. When a code change event arrives via the GitHub webhook, the engine performs a graph traversal to compute the affected test set in under 500 milliseconds for typical repositories.

Data Model: TestDependencyGraph

A new Prisma model **TestDependencyGraph** stores the bidirectional mapping between tests and code elements. Each record links a TestCase to a source file path, function name, or component identifier,

with a confidence score (0.0-1.0) and a last-verified timestamp. The model also captures the dependency type (import, call, render, navigate, or API call) to enable fine-grained impact analysis. A companion model, **SmartSelectionResult**, logs each selection decision, recording which tests were selected, the triggering code change, the selection rationale, and the actual outcomes (pass/fail/skip) for continuous learning and graph refinement.

3.2 Flakiness Prediction and Classification

Flaky tests, those that produce inconsistent results without any code changes, are the single largest source of wasted engineering time in quality assurance. Industry surveys consistently report that 30-50% of engineering teams spend more than 10 hours per week dealing with flaky tests. M29 tackles this problem with a machine learning-based flakiness predictor that classifies each test case into one of four categories: stable (consistently passes), flaky (inconsistent results), failing (consistently fails), and unknown (insufficient data). The classifier uses a rich feature set extracted from historical test runs, including pass rate variance, execution time variance, time-of-day patterns, concurrent execution correlation, and dependency change frequency.

The flakiness classifier operates in two modes: batch analysis and real-time detection. Batch analysis runs nightly, re-evaluating every test case against its full historical run data and updating the flakiness score. Real-time detection monitors test execution streams and raises alerts when a previously stable test shows early warning signs of flakiness, such as increasing execution time variance or a cluster of intermittent failures within a short window. The existing FeatureRiskScore model is extended with flakiness-specific fields, including a numeric flakiness score (0-100), a classification label, the primary flakiness indicator (timing, order-dependency, resource-contention, or external-dependency), and a recommended quarantine action.

Data Model: FlakinessReport

The **FlakinessReport** model stores the per-test flakiness classification, computed by the batch analysis job. Fields include the test case ID, flakiness score (0-100), classification label (stable, flaky, failing, unknown), primary indicator, confidence level, last 10 run outcomes as a JSON array, and the timestamp of the last analysis. A separate **FlakinessAlert** model captures real-time flakiness detection events, linking to the test run that triggered the alert and the specific anomaly detected.

3.3 Impact-Based Test Prioritization

When teams must run a subset of tests due to time constraints, the order in which tests execute matters significantly. Impact-based test prioritization ranks tests so that those most likely to reveal faults run first, maximizing the probability of catching defects early in the test cycle. The prioritization engine uses three signals: code change impact (from the dependency graph), historical failure correlation (tests that have previously failed alongside the changed code), and business criticality (user-defined priority levels for features and test suites). The result is a priority score for each test that determines its execution order.

The prioritization engine integrates with the existing test execution pipeline. When a test run is initiated, the engine queries the dependency graph to find affected tests, applies the flakiness classifier to adjust weights (flaky tests are deprioritized unless explicitly included), incorporates business criticality metadata from the project configuration, and produces an ordered test execution plan. This plan is passed to the test executor agent, which runs tests in the specified priority order. Early termination is supported: if a critical test fails, the system can immediately notify the team rather than completing the entire suite. The impact analysis results are stored in a new **ImpactAnalysisResult** model, providing a full audit trail of prioritization decisions.

3.4 API Routes

Route	Methods	Description
/api/intelligence/dependencies	GET / POST	Query or rebuild test-code dependency graph
/api/intelligence/dependencies/[id]	GET	Get dependency details for a specific test
/api/intelligence/select	POST	Smart test selection based on code changes
/api/intelligence/flakiness	GET	List flakiness reports across all projects
/api/intelligence/flakiness/[id]	GET / PATCH	Get or update a specific flakiness report
/api/intelligence/flakiness/analyze	POST	Trigger batch flakiness analysis job
/api/intelligence/flakiness/alerts	GET	List real-time flakiness alerts
/api/intelligence/prioritize	POST	Generate prioritized test execution plan
/api/intelligence/impact	GET	List impact analysis results
/api/intelligence/impact/[id]	GET	Get specific impact analysis result

Table 2: M29 API Routes

3.5 Credit Actions

Action	Credits	Description
smart_selection	5	Run smart test selection for a code change
flakiness_analysis	10	Run batch flakiness analysis on a project
impact_analysis	20	Run full impact analysis with prioritization
dependency_rebuild	3	Rebuild dependency graph from execution traces

Table 3: M29 Credit Actions

4. M30: Self-Healing Tests v2 and Auto-Maintenance

Phase 1 introduced the auto-heal feature, which suggests fixes when tests fail during execution. M30 dramatically expands this capability into a comprehensive self-healing and auto-maintenance system that proactively repairs tests before they fail, detects deprecating patterns, and keeps test suites healthy without manual intervention. The key evolution from the existing auto-heal is the shift from reactive (fix after failure) to proactive (predict and prevent), and from single-step fixes to multi-step repair chains that can handle complex breakages such as page structure changes, API contract modifications, and authentication flow updates.

4.1 Selector Self-Healing

The most common cause of test failure is selector breakage: a CSS selector, XPath expression, or data-testid that no longer matches the page DOM after a UI change. Selector self-healing addresses this by maintaining multiple candidate selectors for each test step and automatically falling back to alternatives when the primary selector fails. When a test step references a selector that cannot be found, the healing engine opens the page in the sandbox browser, analyzes the current DOM structure, and uses a combination of visual similarity (comparing the expected element screenshot with the current page), structural similarity (matching by tag name, attributes, and sibling relationships), and LLM-assisted semantic matching (understanding the intent of the test step and finding the element that best fulfills it) to locate the correct element.

When a new selector is found, the system generates a repair candidate with a confidence score, applies it to the test step, re-executes the test from that point, and if the test passes, persists the repair as a new primary selector while demoting the old selector to a fallback. The entire process is logged in a **SelectorRepair** model that captures the old selector, new selector, confidence score, DOM snapshot before and after, and the test run that verified the repair. This model extends the existing FixSuggestion model with selector-specific metadata, enabling teams to review and approve automatic repairs or revert them if they introduce false positives.

4.2 Test Code Auto-Maintenance

Beyond selector repairs, M30 introduces comprehensive test code auto-maintenance that handles a broader range of test degradation patterns. The system continuously monitors test suites for four categories of maintenance needs: deprecation warnings (APIs, libraries, or browser features that the test depends on are being phased out), assertion drift (expected values that diverge from actual application behavior due to legitimate feature changes), step sequence staleness (multi-step flows that no longer match the current application navigation), and test code quality issues (duplicate test cases, overly complex assertions, or unused test utilities).

The auto-maintenance engine runs as a periodic analysis job that scans all test cases in a project, cross-references them with recent application changes and deprecation notices from the LLM analysis

agent, and generates maintenance recommendations. Each recommendation includes a severity level (critical, warning, or info), a suggested code change (as a unified diff), an explanation of why the change is needed, and an estimated effort score. Critical recommendations, such as tests that will break due to a known upcoming API deprecation, are automatically promoted to notifications and can optionally trigger automatic repair if the confidence score exceeds a configurable threshold. All maintenance records are stored in a **TestMaintenanceRecord** model, providing a complete history of test health over time.

4.3 Deprecation and Breaking Change Detection

A unique capability of the self-healing system is its ability to detect upcoming deprecations and breaking changes before they cause test failures. The system integrates with GitHub webhook events to monitor for deprecation notices in pull request descriptions, release notes, and changelog files. When a deprecation is detected that affects a test case (determined by the dependency graph from M29), the system generates a proactive maintenance recommendation with a timeline, suggesting that the test be updated before the deprecation becomes effective. For breaking changes that have already been deployed, the system correlates test failures with the code change that caused them and generates a targeted repair suggestion. This forward-looking capability transforms test maintenance from a reactive firefight into a planned, predictable process.

4.4 API Routes

Route	Methods	Description
/api/self-heal/selector-repairs	GET / POST	List or create selector repair records
/api/self-heal/selector-repairs/[id]	GET / PATCH	Get or approve/reject a repair
/api/self-heal/maintenance	GET	List all maintenance recommendations
/api/self-heal/maintenance/[id]	GET / PATCH	Get or action a maintenance record
/api/self-heal/maintenance/scan	POST	Trigger maintenance scan for a project
/api/self-heal/deprecations	GET	List detected deprecations affecting tests
/api/self-heal/deprecations/[id]	GET	Get deprecation details and affected tests
/api/self-heal/auto-repair	POST	Execute automatic repair with confidence threshold

Table 4: M30 API Routes

5. M31: Synthetic Monitoring and Performance Baselines

Testing before deployment is necessary but not sufficient. Many defects only manifest in production: performance regressions under real load, third-party service failures, CDN configuration errors, and DNS resolution problems. M31 extends Probato into production monitoring by introducing synthetic monitoring checkpoints that continuously verify application health from the user perspective, and performance baselines that detect regressions before they impact users. This milestone bridges the pre-deployment and post-deployment quality gap, completing the quality feedback loop.

5.1 Synthetic Monitoring Checkpoints

Synthetic monitoring checkpoints are scheduled, automated browser interactions that run against production URLs at regular intervals to verify that critical user flows remain functional. Unlike the existing Schedule model which runs Playwright test cases on the staging or development environment, synthetic checkpoints execute simplified versions of key user journeys against live production endpoints, measuring not just pass/fail outcomes but also response times, error rates, and content validity. Each checkpoint is defined by a URL, a sequence of interaction steps (navigate, click, type, wait, assert), an expected outcome (status code, content match, visual match), and a schedule (interval in minutes, with a minimum of 5 minutes to respect rate limits and credit budgets).

When a checkpoint fails, the system generates an alert through the existing notification infrastructure, including the specific step that failed, a screenshot of the page at the point of failure, the response time compared to the baseline, and a correlation with recent deployments (from GitHub webhook events). Checkpoints can be configured with different severity levels: critical checkpoints (e.g., login flow, checkout process) trigger immediate alerts to all configured channels, while informational checkpoints (e.g., about page loads, search functionality) generate lower-priority notifications. The **SyntheticCheckpoint** model stores the checkpoint definition, and a companion **CheckpointResult** model records each execution outcome with full telemetry data.

5.2 Performance Baselines and Web Vitals

Performance baselines establish the expected performance characteristics for each synthetic checkpoint and track deviations over time. The system integrates with Lighthouse CI to capture Core Web Vitals metrics (Largest Contentful Paint, First Input Delay, Cumulative Layout Shift) and custom performance metrics (Time to First Byte, DOM Content Loaded, full page load) for each checkpoint execution. Baseline values are computed using a rolling window of the last 30 successful executions, with configurable thresholds for regression detection (default: 20% degradation from baseline triggers a warning, 50% triggers a critical alert).

The performance baseline system introduces two new models. The **PerformanceBaseline** model stores the computed baseline values for each metric associated with a synthetic checkpoint, including the

mean, standard deviation, 50th percentile, 75th percentile, and 95th percentile over the rolling window. The **PerformanceRegression** model records detected regressions, capturing the metric name, baseline value, observed value, deviation percentage, and the checkpoint execution that triggered the detection. Regressions are automatically correlated with deployment events to identify the code change that likely caused the regression, leveraging the dependency graph from M29 to narrow the search scope.

5.3 Integration with Existing Systems

Synthetic monitoring integrates deeply with Probato's existing infrastructure. Checkpoint results feed into the flakiness prediction engine from M29, helping distinguish between genuine production issues and monitoring noise. Performance regressions trigger the self-healing system from M30 when the regression is caused by a selector or assertion change rather than a genuine performance issue. Notification dispatch uses the existing NotificationChannel infrastructure, adding two new event types: checkpoint_failure and performance_regression. Billing integration uses the credit system, with each checkpoint execution consuming credits based on the number of steps and the monitoring interval. The scheduler engine is extended to support high-frequency intervals (down to 5 minutes) for synthetic checkpoints, separate from the existing test schedule system which runs at daily or hourly intervals.

5.4 API Routes

Route	Methods	Description
/api/monitoring/checkpoints	GET / POST	List or create synthetic checkpoints
/api/monitoring/checkpoints/[id]	GET / PATCH / DELETE	Manage a specific checkpoint
/api/monitoring/checkpoints/[id]/results	GET	Get execution results for a checkpoint
/api/monitoring/baselines	GET	List performance baselines
/api/monitoring/baselines/[id]	GET / PATCH	View or adjust baseline thresholds
/api/monitoring/regressions	GET	List detected performance regressions
/api/monitoring/regressions/[id]	GET	Get regression details and correlation
/api/monitoring/dashboard	GET	Aggregated monitoring dashboard data

Table 5: M31 API Routes

6. M32: Enterprise SSO, Audit, and Compliance

Enterprise adoption requires more than just features; it demands governance, security, and compliance capabilities that allow organizations to trust the platform with their code, test data, and internal workflows. M32 delivers the three critical enterprise requirements: Single Sign-On (SSO) integration via SAML and OpenID Connect, comprehensive audit logging for regulatory compliance, and role-based access control enhancements that support the complex permission structures of large organizations. These capabilities are essential for Probato to penetrate the enterprise market segment, which represents the highest revenue potential for the platform.

6.1 Single Sign-On Integration (SAML and OIDC)

The SSO integration enables organizations to manage Probato access through their existing identity providers, including Okta, Azure Active Directory, Google Workspace, and OneLogin. The implementation supports both SAML 2.0 and OpenID Connect (OIDC) protocols, with SAML as the primary protocol for enterprise customers and OIDC as a lighter-weight alternative for organizations that prefer OAuth 2.0-based authentication. The SSO flow is implemented as a NextAuth.js custom provider that delegates authentication to the configured identity provider, receives the authentication response, and maps identity provider groups and roles to Probato team roles and permissions.

The SSO configuration is managed at the team level, allowing each team to configure its own identity provider connection. The **SSOConfiguration** model stores the protocol type (SAML or OIDC), the identity provider metadata (entity ID, SSO URL, certificate for SAML; client ID, issuer URL, discovery URL for OIDC), the group-to-role mapping rules, and the domain restrictions (which email domains are allowed to authenticate via this SSO configuration). When a user authenticates via SSO, the system automatically creates or updates their Probato account, assigns them to the appropriate team based on their identity provider groups, and applies the corresponding role permissions. This eliminates the need for manual user provisioning and ensures that access changes in the identity provider are immediately reflected in Probato.

6.2 Comprehensive Audit Logging

Audit logging provides a tamper-evident record of all significant actions performed on the platform, enabling organizations to meet regulatory requirements such as SOC 2 Type II, GDPR, HIPAA, and ISO 27001. The audit log captures who performed each action, what the action was, when it occurred, from where (IP address and user agent), and what resources were affected. Every API endpoint that modifies data is instrumented to emit audit log entries, including test execution, project creation and deletion, team membership changes, billing modifications, SSO configuration updates, and API key operations.

The **AuditLog** model stores each audit entry with an immutable hash chain that links consecutive entries, making it possible to detect any tampering with the log. Each entry includes the actor (user ID or system service), action type (create, read, update, delete, execute), resource type and ID, before and after

snapshots (for update actions), the IP address, user agent, and a SHA-256 hash of the previous entry. A companion **AuditLogExport** model supports scheduled exports of audit logs to external SIEM systems (Splunk, Datadog, AWS CloudTrail) via webhook delivery, enabling integration with existing enterprise security monitoring infrastructure. The audit log retention policy is configurable per team, with a default of 90 days and options for 180 days, 1 year, or indefinite retention for regulated industries.

6.3 Role-Based Access Control v2

The existing team role system (owner, admin, member, viewer) is extended with granular permissions that enable fine-grained access control at the project, feature, and test level. The enhanced RBAC system introduces four concepts: permission policies (named sets of permissions that can be assigned to roles), resource scopes (which resources a permission applies to), permission conditions (contextual rules such as time-of-day restrictions or IP allow-lists), and delegated administration (allowing non-owner users to manage specific resources). The system ships with five predefined permission policies matching the existing role structure, plus the ability to create custom policies for organizations with unique compliance requirements.

The implementation extends the existing TeamMember model with a **PermissionPolicy** reference and a JSON field for custom permission overrides. A new **ResourcePermission** model captures project-level and test-level permission grants that override the team-level policy. The permission evaluation engine is implemented as middleware that intercepts API requests, resolves the user's effective permissions by merging their team policy with any resource-specific overrides, and checks whether the requested action is permitted. All permission evaluations are logged to the audit system, providing complete visibility into access decisions. This architecture supports the principle of least privilege while maintaining the simplicity of the current role system for teams that do not need granular permissions.

6.4 API Routes

Route	Methods	Description
/api/sso/config	GET / POST	Get or create SSO configuration for a team
/api/sso/config/[id]	GET / PATCH / DELETE	Manage SSO configuration
/api/sso/metadata	GET	Get SP metadata for IdP configuration
/api/sso/callback	POST	SSO authentication callback endpoint
/api/audit/logs	GET	Query audit log entries with filters
/api/audit/logs/[id]	GET	Get specific audit log entry

/api/audit/exports	GET / POST	List or create audit log export configurations
/api/audit/exports/[id]	GET / PATCH / DELETE	Manage export configuration
/api/audit/verify	POST	Verify audit log hash chain integrity
/api/permissions/policies	GET / POST	List or create permission policies
/api/permissions/policies/[id]	GET / PATCH / DELETE	Manage a permission policy
/api/permissions/check	POST	Check if a user has a specific permission

Table 6: M32 API Routes

7. M33: Plugin Architecture and Integrations Marketplace

No platform can anticipate every testing need. M33 opens Probato to community and third-party innovation by introducing a plugin architecture that allows developers to extend the platform with custom test types, result processors, notification channels, and data source integrations. The plugin system is designed with security as a primary concern: plugins run in sandboxed execution contexts, have explicitly declared permissions, and are subject to code review before publication in the integrations marketplace. This milestone transforms Probato from a closed platform into an extensible ecosystem, enabling rapid innovation without compromising platform stability or security.

7.1 Plugin SDK and Runtime

The Plugin SDK provides a TypeScript API for developing Probato plugins. Plugins are packaged as npm modules with a specific manifest file (probato-plugin.json) that declares the plugin's name, version, description, permissions, and extension points. The SDK defines five extension point types: test executors (custom test runners that implement the ITestExecutor interface), result processors (post-processing hooks that analyze test results and generate insights), notification channels (new delivery methods for alerts), data sources (integrations with external tools such as Jira, Linear, or PagerDuty), and UI panels (custom dashboard components rendered in the Probato UI via a secure iframe sandbox).

The plugin runtime manages the lifecycle of installed plugins, including installation, activation, configuration, execution, and deactivation. Plugins are executed in isolated V8 isolates (using Node.js worker threads) with resource limits (CPU time, memory, and network access) enforced by the runtime. The runtime also provides a messaging API that allows plugins to communicate with the core platform through well-defined interfaces, preventing direct database access and ensuring that all plugin operations

are logged and auditable. The **Plugin** model stores the plugin manifest, installation status, configuration schema, and the team that installed it. The **PluginExecution** model records each plugin invocation with input parameters, output results, resource consumption metrics, and any errors encountered.

7.2 Integrations Marketplace

The Integrations Marketplace is a curated directory of verified plugins that teams can browse, install, and configure directly from the Probato dashboard. The marketplace supports three tiers of plugins: official plugins (built and maintained by the Probato team, such as Jira integration, Slack advanced notifications, and Datadog metrics export), verified plugins (built by third parties and reviewed by the Probato team for security and quality), and community plugins (published by any developer without formal review, available with an explicit warning). Each marketplace listing includes a description, screenshots, permission requirements, installation count, average rating, and a link to the source code repository.

The marketplace is implemented as a new section in the Probato dashboard, with API endpoints for browsing, searching, installing, and rating plugins. Plugin installation is a two-step process: first, the team admin reviews the plugin permissions and approves the installation; second, the plugin runtime downloads the plugin package, validates its integrity (using SHA-256 checksums and digital signatures), and registers it with the platform. Installed plugins appear in a dedicated management panel where administrators can configure settings, view execution logs, and deactivate or uninstall plugins. The marketplace UI also supports private plugins, which are hosted on the team's own infrastructure and not visible to other teams, enabling organizations to build internal integrations without publishing them publicly.

7.3 API Routes

Route	Methods	Description
/api/plugins	GET / POST	List installed plugins or upload a new plugin
/api/plugins/[id]	GET / PATCH / DELETE	Manage a specific plugin
/api/plugins/[id]/configure	POST	Update plugin configuration
/api/plugins/[id]/activate	POST	Activate an installed plugin
/api/plugins/[id]/deactivate	POST	Deactivate a plugin
/api/plugins/[id]/executions	GET	List plugin execution history
/api/marketplace	GET	Browse marketplace listings
/api/marketplace/[id]	GET	Get marketplace listing details
/api/marketplace/[id]/install	POST	Install a marketplace plugin

/api/marketplace/[id]/reviews	GET / POST	List or submit plugin reviews

Table 7: M33 API Routes

8. M34: Phase 6 Integration and Polish

The final milestone of Phase 6 focuses on cross-feature integration, SDK updates, documentation, performance optimization, and quality assurance. While each milestone delivers standalone value, the true power of Phase 6 emerges when the intelligence engine, self-healing system, synthetic monitoring, enterprise features, and plugin architecture work together as a unified platform. M34 ensures that these capabilities are deeply integrated, well-documented, performant, and thoroughly tested before the phase is declared complete.

8.1 Cross-Feature Integration Workstreams

The integration workstreams connect the Phase 6 features into coherent end-to-end workflows. The first workstream, **Intelligence-to-Action Loop**, connects M29 and M30 so that flakiness predictions automatically trigger self-healing actions, and impact analysis results inform selector repair prioritization. When the intelligence engine identifies a flaky test, it not only classifies the flakiness but also suggests the most likely cause (e.g., a selector that depends on dynamic content), and the self-healing engine uses this hint to generate more targeted repair candidates. The second workstream, **Test-to-Monitor Pipeline**, connects M29 and M31 by allowing teams to promote any test case to a synthetic monitoring checkpoint with a single click, automatically configuring the checkpoint schedule, step sequence, and expected outcomes from the test definition. The third workstream, **Compliance-to-Audit Trail**, ensures that all Phase 6 actions (intelligence queries, self-healing repairs, monitoring alerts, plugin executions) generate audit log entries, providing a complete compliance picture.

8.2 SDK and API Documentation Updates

The Probato SDK is extended with four new resource classes corresponding to the Phase 6 features. The **Intelligence** resource provides methods for smart test selection, flakiness analysis, and impact prioritization. The **SelfHeal** resource exposes selector repair and maintenance scan operations. The **Monitoring** resource covers synthetic checkpoint management and performance baseline queries. The **Plugins** resource enables programmatic plugin installation and configuration. Each resource follows the existing SDK pattern of HTTP client methods with TypeScript type definitions, error handling, and rate limit awareness. The OpenAPI specification is updated with all new endpoints, and the interactive API documentation at /api/v1/docs is regenerated to include Phase 6 endpoints with request/response examples.

8.3 Performance Optimization and Quality Assurance

Phase 6 introduces several computationally intensive operations (dependency graph traversal, flakiness analysis, selector healing DOM analysis, plugin sandboxing) that require careful performance optimization. M34 includes a dedicated performance audit that benchmarks all new API endpoints against target response times: dependency graph queries under 500ms, flakiness analysis under 30 seconds for a typical project, selector healing under 10 seconds per repair, and synthetic checkpoint execution under 60 seconds. Database query optimization includes adding indexes for the new models, materialized views for frequently queried aggregations (such as flakiness scores and performance baselines), and connection pooling configuration for high-concurrency monitoring workloads. The test suite is expanded with integration tests that exercise the cross-feature workflows, edge case tests for the plugin sandbox, and load tests for the synthetic monitoring system. Target: 700+ total tests passing across all phases.

9. Data Model Summary

Phase 6 introduces 12 new Prisma models that extend the existing 42-model schema. These models are designed to integrate with the existing User, Project, TestCase, TestRun, and Team models through foreign key relationships, maintaining referential integrity and enabling efficient cross-feature queries. The following table summarizes all new models with their primary purpose and the milestone that introduces them.

Model	Milestone	Purpose
TestDependencyGraph	M29	Bidirectional mapping between tests and code elements
SmartSelectionResult	M29	Log of smart test selection decisions
FlakinessReport	M29	Per-test flakiness classification and scores
FlakinessAlert	M29	Real-time flakiness detection events
ImpactAnalysisResult	M29	Impact analysis and prioritization records
SelectorRepair	M30	Selector self-healing repair records
TestMaintenanceRecord	M30	Test code maintenance recommendations
SyntheticCheckpoint	M31	Synthetic monitoring checkpoint definitions
CheckpointResult	M31	Checkpoint execution outcomes and telemetry
PerformanceBaseline	M31	Computed performance metric baselines
PerformanceRegression	M31	Detected performance regression events
SSOConfiguration	M32	SAML/OIDC identity provider configurations
AuditLog	M32	Tamper-evident audit log entries

AuditLogExport	M32	Scheduled audit log export configurations
PermissionPolicy	M32	Granular permission policy definitions
ResourcePermission	M32	Resource-level permission overrides
Plugin	M33	Installed plugin manifest and configuration
PluginExecution	M33	Plugin invocation history and metrics
MarketplaceListing	M33	Marketplace plugin directory entries

Table 8: Phase 6 New Data Models

10. Implementation Timeline and Dependencies

Phase 6 milestones have carefully managed dependencies. M29 (AI Test Intelligence Engine) must be completed before M30 (Self-Healing Tests v2) because the self-healing system uses dependency graph data and flakiness predictions to generate more accurate repairs. M31 (Synthetic Monitoring) depends on M29 for checkpoint promotion from test cases and for correlating monitoring results with deployment events. M32 (Enterprise SSO and Audit) is independent of M29-M31 and can be developed in parallel. M33 (Plugin Architecture) depends on M32 for the permission model that governs plugin access. M34 (Integration and Polish) depends on all preceding milestones. The following timeline assumes sequential development with M32 developed in parallel with M29-M31.

Milestone	Depends On	Estimated Duration	Parallel Track
M29: AI Test Intelligence	Phase 5 complete	2-3 weeks	Track A
M30: Self-Healing v2	M29	2-3 weeks	Track A
M31: Synthetic Monitoring	M29	2-3 weeks	Track A
M32: Enterprise SSO/Audit	Phase 5 complete	2-3 weeks	Track B
M33: Plugin Architecture	M32	2-3 weeks	Track B
M34: Integration and Polish	M29-M33	1-2 weeks	Final

Table 9: Phase 6 Implementation Timeline

11. Credit and Billing Impact

Phase 6 introduces several new credit-consuming actions that expand the existing billing model. The credit costs are calibrated to be proportional to the computational resources consumed and the business value delivered. Intelligence and analysis operations (M29) are priced higher than simple test executions because they involve LLM calls and graph computations. Self-healing repairs (M30) are priced comparably to the existing auto-heal action. Synthetic monitoring (M31) uses a per-check credit model

that accounts for both the browser execution time and the monitoring frequency. Enterprise features (M32) and the plugin marketplace (M33) are plan-level features rather than credit-consuming actions, incentivizing upgrades to Team and Enterprise plans.

Action	Credits	Plan Availability
smart_selection	5	Pro, Team, Enterprise
flakiness_analysis	10	Pro, Team, Enterprise
impact_analysis	20	Team, Enterprise
dependency_rebuild	3	Pro, Team, Enterprise
selector_repair	8	Pro, Team, Enterprise
maintenance_scan	6	Pro, Team, Enterprise
synthetic_checkpoint	3	Team, Enterprise
performance_baseline	2	Team, Enterprise
sso_authentication	N/A	Enterprise plan feature
audit_log_access	N/A	Team, Enterprise plan feature
plugin_install	Variable	All plans (marketplace-dependent)

Table 10: Phase 6 Credit Actions and Plan Availability

The plan tier restrictions for Phase 6 features are designed to drive upgrades while maintaining accessibility for individual developers. Smart test selection and flakiness analysis are available on the Pro plan, ensuring that individual professionals and small teams can benefit from AI intelligence. Impact analysis, synthetic monitoring, and performance baselines require the Team plan, reflecting the collaborative and production-facing nature of these features. SSO and advanced audit logging are Enterprise-only features, consistent with the typical enterprise pricing model. Plugin installation is available on all plans, but some marketplace plugins may have their own plan requirements or separate pricing.

12. Risks and Mitigations

Phase 6 introduces several technical and business risks that must be proactively managed. The following analysis identifies the most significant risks and proposes concrete mitigation strategies for each.

Risk	Severity	Mitigation
Dependency graph accuracy	High	Continuous learning from selection outcomes; fallback to full suite when confidence < 80%

Self-healing false positives	High	Confidence thresholds; human approval for critical tests; repair revert capability
Synthetic monitoring cost	Medium	Credit budgets per checkpoint; frequency caps; efficient headless execution
Plugin sandbox escape	Critical	V8 isolate sandboxing; resource limits; security review for verified plugins
SSO integration complexity	Medium	Support top 3 IdPs first (Okta, Azure AD, Google); community templates for others
Audit log performance	Medium	Async log writing; batch exports; time-based partitioning for large volumes
Flakiness classifier accuracy	Medium	Retrain on false positive feedback; ensemble methods; human override labels

Table 11: Phase 6 Risk Assessment

The most critical risk is plugin sandbox escape, which could allow a malicious plugin to access sensitive data or execute arbitrary code on the server. This risk is mitigated by a defense-in-depth strategy: plugins run in isolated V8 contexts with no direct access to the filesystem, network, or database; all plugin operations go through the platform API which enforces permission checks; verified plugins undergo manual code review; and the plugin runtime enforces strict resource limits (CPU time, memory, and network requests). Additionally, the audit log captures all plugin operations, enabling rapid detection and investigation of any suspicious activity. If a sandbox escape vulnerability is discovered, the plugin can be immediately deactivated across all installations through the marketplace infrastructure.